ANANTA HACKATHON 2026 · TEAM 11 · PS 11

ECOGUARD AI

How it works

A walk through the whole prototype: the building we simulate, every waste we look for, the model, the checks for hidden waste, how it acts, and what it saved.

For	Our team, mentor and jury questions
Code	ecoguard/ folder in our repository (README has the run steps)
Result	19.2% less electricity, 22.6% less water, ₹46,380 a year saved in one block, 3.6 t CO_2 avoided
Detection	recall 95.2%, precision 99.2% on a year the model never saw

1. What changed since Round 1

Our Round 1 deck promised ECOGUARD for every building and every kind of waste. The jury found it too broad, and our mentor gave us a clear direction in the 14 November meeting:

- **Pick one environment.** We chose one academic block: five classrooms, a computer lab and two offices. Everyone on the jury has sat in these rooms.
- **Cover every connected waste inside it.** Electricity and water together, including the pump that links them, and the waste nobody can see.
- **Simulate it and show before vs after.** One full year, same events, with and without ECOGUARD, in kWh, litres and rupees.
- **Train on data.** We generated a labelled year from public numbers, trained on it, and tested on a second year the model had never seen.
- **Point to the exact problem.** Every alert names the room, the cause, the cost and the next step.

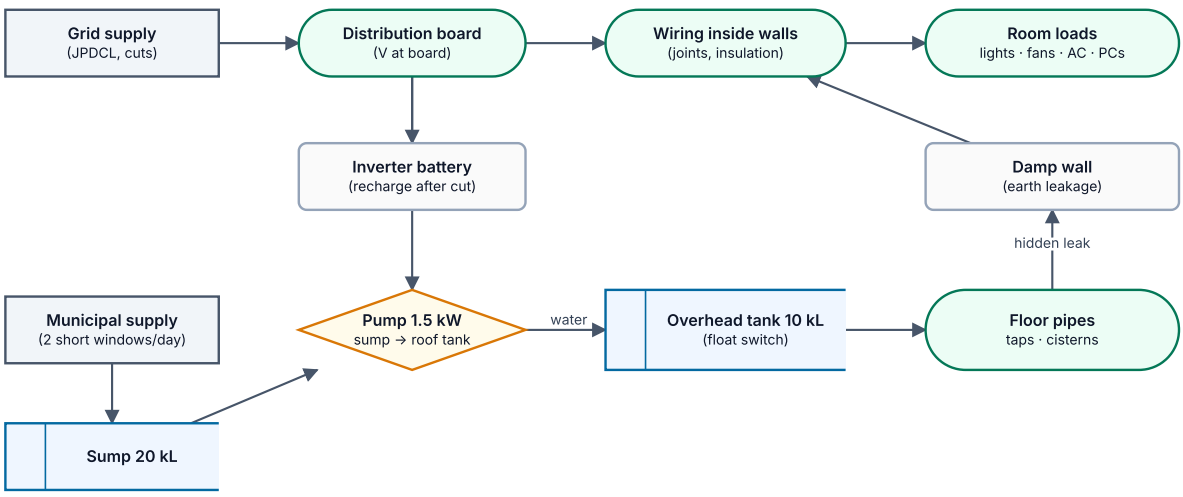
The core idea from Round 1 stays the same: learn what normal looks like, flag what is not, explain it in plain words and act on it. What changed is focus and proof.

2. The building: Academic Block A, Jammu

ID	Room	Floor	Seats	Equipment	Inverter	Wire R (Ω)
OFF-ADM	Admin Office	Ground	8	1 AC, 8 PCs, heater, 4 fans	yes	0.16
LAB-1	Computer Lab	Ground	40	3 AC, 40 PCs, 6 fans	no	0.14
CR-101	Classroom 101	First	60	8 fans	no	0.22
CR-102	Classroom 102	First	60	8 fans	no	0.24
CR-103	Seminar Room 103	First	90	2 AC, 6 fans	no	0.2
CR-201	Classroom 201	Second	60	1 AC, 8 fans	no	0.27
CR-202	Classroom 202	Second	60	8 fans	no	0.29
OFF-FAC	Faculty Room	Second	10	2 AC, 10 PCs, heater, 5 fans	yes	0.25

Water: a 20 kL sump fed by municipal supply in two short windows a day, a 1.5 kW pump (150 L/min) lifting to a 10 kL roof tank that feeds three floors of washrooms. Prices: ₹7/kWh, ₹30/kL. Carbon: 0.716 kg CO_2 per kWh (CEA).

3. Every waste in the block, and how they connect



Electricity flows left to right along the top; water along the bottom. The pump sits where they meet. A hidden leak in a wall also shows up on the electrical side as earth leakage.

Waste	What happens	How we see it	What we do
Lights, fans, AC left on	Class ends, room empties, loads keep running	People = 0 but power above the room's expected use	Switch room off after 15 min
Lab PCs on overnight	40 PCs idle all night	Same, at night	Switch off
Comeback after power cut	Switches were left on; power returns to an empty room	Empty-room use within an hour of restore	Switch off
AC set too cold	Remote set to 16-18 °C	Room below 21.5 °C with the AC running	Reset AC to 24 °C
AC fighting an open door	Door propped open in May	AC flat out, room still above 27 °C	Alert the room
Heat in the walls	Loose or corroded joint gets hot	Wiring resistance rising over days	Electrician ticket
Earth leakage	Damp wall or worn insulation	> 15 mA for 2 hours on a floor	Electrician ticket
Pipe leak	Water lost all day and night	Night flow never drops to zero	Close floor valve + plumber
Running cistern / drip	Small but endless flow	Low steady night flow	Plumber ticket
Tank overflow	Float switch stuck; pump keeps lifting water off the roof	Pump on, tank full	Stop pump, level interlock
Pump dry run	Sump empty, pump spins with no water	Pump on, no flow	Stop pump

4. The simulator

The simulator (`sensors/sim/`) moves the block forward in 15-minute steps. Each step it works out:

- **Weather** from IMD climate normals for Jammu, with day-to-day noise, so summers are hot and winters need heaters.
- **People** from a weekly timetable, with holidays, semester breaks, cancelled classes and people staying late. The timetable is what is planned; the headcount is what really happened.
- **Indoor climate**: each room's temperature and humidity follow the weather, the people inside and the AC (a first-order thermal model). An AC set to 17 °C pulls the room down to 17 °C; an open door stops it reaching 24 °C.
- **Room power** from the equipment list: lights, fans, ACs that work harder on hot days, PCs, heaters and standby loads.
- **Power cuts** in the JPDCL pattern, with voltage sag at peak hours, and inverter batteries that recharge once power returns.
- **Wiring**: each circuit has a resistance, so the room sees a lower voltage than the board. A loose joint raises that resistance slowly.
- **Water**: use per person per hour, cleaning in the morning, municipal supply windows, the sump, the pump and the roof tank.
- **Faults**: eleven kinds, started at random but at realistic rates (lights left on is common; a failing joint is rare). Each fault keeps a record of exactly how much it wasted. That record is the ground truth.

> Every step is driven by a seeded random generator. With the same seed you get exactly the same year. That is what makes a fair before-vs-after test possible: the world is identical, only ECOGUARD's presence changes.

The gateway (`sensors/gateway.mjs`) turns each step into the same JSON message a real ESP32 node would send, posts it to the backend, and applies any command that comes back (switch off, stop pump, close valve, repair ticket).

5. The model: what should this room be using?

For each room we train one regression model that predicts power from:

- whether the room is in use, and how many people are inside
- cooling need above 25 °C and above 30 °C, and heating need below 13 °C
- whether it is daylight
- the hour of the day when the room is empty (24 small terms, to learn standby and fridge-like loads)

Training uses one simulated year (seed 11) up to 31 March; the rest is held out. We use **Huber regression**: it behaves like a normal straight-line fit for ordinary readings but stops large spikes (the waste already hiding in the training data) from dragging the line upwards. The spread of normal readings is measured with the median absolute deviation, which is also not fooled by spikes.

Room	Error, Huber (kW)	Error, plain least squares (kW)	Average use (kW)
CR-101	0.024	0.046	0.138
CR-102	0.051	0.067	0.158
CR-103	0.086	0.147	0.236
CR-201	0.143	0.186	0.329
CR-202	0.031	0.066	0.148
LAB-1	0.317	0.525	0.878
OFF-ADM	0.089	0.142	0.723
OFF-FAC	0.192	0.262	1.137

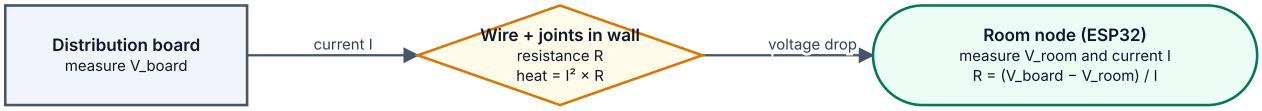
Mean absolute error on held-out data. Huber wins in every room.

6. The detector

Check	Rule
Empty room	People = 0 and use above expected by more than max(0.2 kW, 4σ), for 30 minutes
Over use	Occupied and use above expected by more than max(0.6 kW, 6σ) and 35%, for one hour
AC set too cold	Occupied, AC drawing power and the room below 21.5 °C for 30 minutes (wall sensor + AC current clamp)
AC fighting an open door	Occupied, AC at 95% of its rating or more, room still above 27 °C and above 30 °C outside, for one hour
Heat in the walls	Wiring resistance R = (V_board − V_room) / I, median of the last 6 hours, above 1.6 × the circuit's own normal and at least 0.1 Ω higher
Earth leakage	Worst circuit on a floor above 15 mA for 2 hours (an RCCB trips at 30 mA)
Night flow	Lowest flow on a floor over 3 empty hours above 6 L/h. Above 40 L/h it is a leak; with earth leakage on the same floor, it is a leak inside the wall
Overflow	Pump running while the tank is at 99.5% or more
Dry run	Pump running with no water coming out, for 30 minutes

After a power cut, rooms on an inverter are allowed about 0.65 kW extra for 8 hours: that is the battery recharging, not waste. Energy checks pause while the grid is off, because the meters read zero.

7. Heat and damp in the walls (our novel check)



Wiring is never perfect. A healthy classroom circuit here has about 0.14 to 0.29 Ω between the board and the room. When a joint loosens, oxidises or a wire is undersized, that number climbs, and every amp that flows through it turns into heat inside the wall: **$P = I^2 \times R$**. At 15 A, an extra 0.3 Ω means about 70 W of heat in one spot, day after day. No bill shows it and no meter separates it, and it is how many electrical fires start.

We need only two voltage readings and one current reading, which the room node already has. ECOGUARD learns each circuit's normal resistance from its first readings and then watches the trend. In the test year it caught all 8 failing joints, typically about 9 days after they started, well before they would heat up enough to fail.

The second hidden check links water and wiring: a pipe leaking inside a wall makes it damp, and a damp wall leaks current to earth. Night water flow plus rising earth leakage on the same floor tells the plumber to look inside the wall, not just at the taps.

8. Acting on what it finds

- **Automatic mode** (on by default): after a 15-minute grace period the system acts. Rooms are switched off only when nobody is inside, and only non-essential loads. The pump is stopped on overflow or dry run, and the level sensors then control it so a stuck float switch no longer matters. A floor valve is closed only for a real leak (above 40 L/h), never for a drip.
- **Repair tickets** go out at once for hidden waste, leaks and pump faults, with a due time (24 hours by default).
- **People stay in charge.** "Not waste" on any alert stops the action and is saved as a label. Automatic mode can be turned off in Settings.
- **Advice** is written in English and Hindi from templates. The numbers in it (kW, litres, rupees, CO_2) are always computed in code. If we add a language model later, it only rephrases; it never makes up a number.
- **Forecast**: the next 24 hours of use from the timetable and the weather, so the manager sees tomorrow's expected bill and peak hour.

9. Results

Detection, on a year the model never saw (seed 99)

558 faults that actually wasted something; 1205 alerts raised. **Recall 95.2%** (faults caught), **precision 99.2%** (alerts that were real).

Fault	Count	Caught	Recall	Typical delay
Lights and fans left on in empty room	245	224	**91%**	15 min
Loads restarted in empty room after power cut	122	120	**98%**	15 min
AC left running after class	64	63	**98%**	15 min
AC set too cold (16-18 deg setpoint)	31	29	**94%**	30 min
Lab PCs left on overnight	31	31	**100%**	15 min
Running toilet cistern	21	21	**100%**	2.8 h
Pipe leak (possibly inside wall)	16	16	**100%**	3.0 h
Tank overflow: float switch stuck	9	9	**100%**	0 min
Heat in the wall: loose joint / failing wire	8	8	**100%**	8.6 days
Pump running dry (sump empty)	6	6	**100%**	11.9 h
AC fighting an open door or window	5	4	**80%**	45 min

Alert type	Alerts	Real	Precision
door_open	4	4	**100%**
Pump running dry	18	18	**100%**
Current leaking to earth	14	14	**100%**
Power used in an empty room	587	587	**100%**
Water flowing in an empty building	239	239	**100%**
Using much more than expected	2	0	**0%**
overcool	31	31	**100%**
Tank overflowing	18	18	**100%**
Heat in the wall wiring	292	284	**97%**

Before vs after: one full year, same block

Same people, same weather, same power cuts and the same 520 faults, run twice from 2025-07-07: once with nobody watching, once with ECOGUARD connected through the real backend.

	Without ECOGUARD	With ECOGUARD	Saved
Electricity	26,000 kWh	21,019 kWh	**4,981 kWh (19.2%)**
Water	1,680 kL	1,300 kL	**380 kL (22.6%)**
Pump electricity	1,140 kWh	327 kWh	813 kWh
Bill	₹232,395	₹186,142	**₹46,253**
CO_2	18.6 t	15.1 t	**3.6 t**
Rooms switched off with people inside		0	

Waste by type

Waste	Times	Before	After	Cut
Lab PCs left on overnight	26	1,482.3 kWh	48.7 kWh	**97%**
Lights and fans left on in empty room	219	1,416.0 kWh	144.6 kWh	**90%**
AC left running after class	66	1,016.9 kWh	99.7 kWh	**90%**
Loads restarted in empty room after power cut	133	511.9 kWh	134.8 kWh	**74%**
AC set too cold (16-18 deg setpoint)	14	52.4 kWh	21.2 kWh	**60%**
Room heater left on	1	30.5 kWh	1.5 kWh	**95%**
Heat in the wall: loose joint / failing wire	7	43.2 kWh	0.7 kWh	**98%**
Pump running dry (sump empty)	14	422.9 kWh	0.0 kWh	**100%**
Tank overflow: float switch stuck	14	154.4 kL	4.2 kL	**97%**
Pipe leak (possibly inside wall)	8	216.2 kL	3.4 kL	**98%**
Running toilet cistern	15	23.0 kL	6.4 kL	**72%**

Scaled up: one block saves about ₹46,380 a year. A campus with 20 similar blocks would save around ₹9.3 lakh and 71 t CO_2 a year.

10. Real buildings: trained and tested on real meters

To prove the models on real data, we used the **Building Data Genome 2** (BDG2) open data set (Miller et al., Scientific Data, 2020): 1,636 real non-residential buildings with two full years of hourly meter readings and on-site weather. We kept the ones that match our block: **242 classroom buildings, 143 labs and 274 offices**.

Model	What it does	Result on 2017 (never seen)
Electricity baseline LightGBM, one model for all buildings	Predicts each building's normal hourly use from hour, weekday, holidays, weather, size and its own typical profile. Trained on 5.5 million hourly rows.	Median CV(RMSE) 20.8%, bias -0.6%; 57.1% of buildings pass ASHRAE Guideline 14
Cooling (HVAC) baseline LightGBM on chilled-water meters	The same idea for cooling energy in 340 buildings.	Median CV(RMSE) 47.9% (cooling is spikier and harder)
Day-ahead forecaster 2-layer LSTM + MLP, PyTorch	Reads the last 7 days and predicts the next 24 hours, for planning pre-cooling and the next day's bill.	Median error 14.1% vs 22.0% for 'same as yesterday' and 19.5% for 'same as last week'; beats both in 92.3% of buildings

Waste found in real buildings

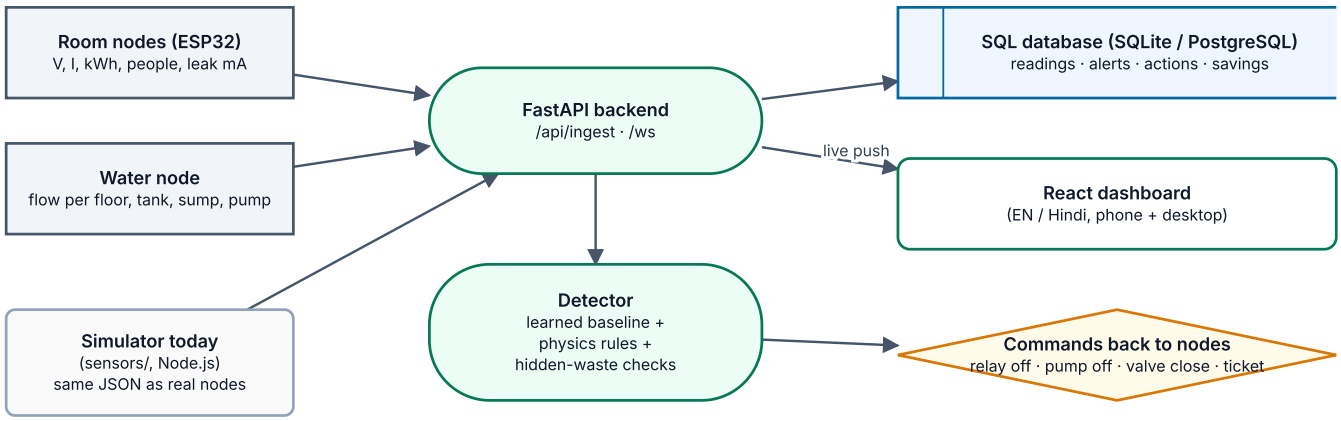
Check	Finding
Avoidable electricity (after-hours above the building's own best nights, plus waste events)	Median 13.8% of the year's use; one building in four above 24.9%. Classrooms 18.8%, offices 14.2%, labs 6.5%.
Cooling while closed (HVAC)	Median 43.0% of chilled-water energy is used when the building is closed (332 buildings).
Heating and cooling at the same time	47 buildings for more than 100 hours in the year.
Water: minimum night flow	20 of 57 water-metered buildings show runs of 3+ nights with leak-like flow.

The dashboard's **Real Buildings** page lets anyone open 38 of these buildings, replay 2017 week by week against the model, see every waste event on a year heatmap, and move sliders (switch-off when closed, act on waste events, schedule HVAC) to see what ECOGUARD would have saved, in kWh, rupees at the Jammu tariff and tonnes of CO_2.

11. Every module in our Round 1 deck, working

Deck module	How it works in the prototype
Computer vision occupancy (OpenCV, YOLO)	The Camera Occupancy page and `cv/camera_node.py` count people with a YOLO11 person detector on the server. The frame is processed in memory and thrown away; only the count is stored (DPDP Act data minimisation).
Electricity, water and environment sensors	Room nodes send voltage at both ends of the circuit, current, earth leakage, indoor temperature and humidity, and AC power; the water node sends flow per floor, tank and sump levels and pump state.
MQTT	Nodes publish to `ecoguard/block-a/readings` and get commands on `ecoguard/block-a/commands`. HTTP works too.
Anomaly detection, forecasting, cross-utility	Learned baseline + physics rules + hidden-waste checks; LSTM forecaster; pump and damp-wall checks link energy and water.
HVAC optimisation	HVAC & Comfort page: indoor temperature against the 23-27 °C comfort band, AC load, one-tap reset to 24 °C (the BEE default), open-door detection.
Automatic, manual and scheduled modes	Automatic (after a 15-minute grace period), manual buttons on every alert, and a scheduled closing-time shutdown of rooms that are already empty.
Voice commands	Microphone button in the header (English or Hindi): "switch off classroom 202", "status of the lab", "open alerts". It refuses to switch off a room with people in it.
Fail-safe local control	If the server or network is down, the nodes switch off rooms empty for an hour and run the pump from the level sensors on their own.
Alerts with type, time, severity, status	Every alert has a severity (high for fire, flood or pump-damage risks), a status, advice in English and Hindi, and two feedback buttons that build a labelled data set.
Savings and carbon	Live savings from actions, plus the full-year before vs after and the real-building sandbox.

12. Architecture



Part	Built with
Room and water nodes	ESP32 with current, voltage, leakage and flow sensors (simulated today by `sensors/`, same message format)
Backend	Python, FastAPI, SQLAlchemy; SQLite on a laptop, PostgreSQL/TimescaleDB on a server; WebSocket for live updates
Model training	pandas, scikit-learn (`ml/train_baseline.py`, `ml/evaluate.py`); LightGBM and PyTorch for the real buildings (`ml/real/`); YOLO11n for camera counts (`cv/`)
Messaging	HTTP by default; MQTT (`ecoguard/block-a/readings` and `/commands`) when `MQTT_URL` is set
Dashboard	React, Tailwind CSS, Recharts; twelve screens, English and Hindi, voice commands, works on phones

Database tables

buildings · zones · devices · occupancy_records · energy_readings · water_readings · alerts (with feedback label) · control_actions · savings_daily · users · settings

Main API

Endpoint	Purpose
`POST /api/ingest`	A node sends readings; the reply carries any commands for it
`GET /api/live`, `/ws`	Current state of every room, the tank and the pump
`GET /api/alerts`	Open and past alerts with advice, cost and cause
`POST /api/alerts/{id}/confirm | dismiss`	The two feedback buttons
`GET /api/labels.csv`	All labelled alerts, for retraining

`POST /api/control`	Manual switch off, pump off, valve close, ticket
`GET /api/wiring`, `/api/water/series`, `/api/forecast`, `/api/savings`	Data for each screen
`GET/PUT /api/settings`	Automatic mode, grace period, tickets, language
`POST /api/sim`	Demo only: inject a fault or a power cut

13. Running the demo

1. `cd backend && pip install -r requirements.txt && uvicorn app.main:app --port 8000`
2. `cd frontend && npm install && npm run build` (the backend then serves the dashboard at `localhost:8000`)
3. `cd sensors && node gateway.mjs --api http://localhost:8000 --speed 2 --start 2026-04-13`

Five-minute demo script

1. **Overview**: the building map. Point out a room that is empty but drawing power.
2. **Simulator**: inject "AC left on" in CR-201. Back on Overview the room turns amber and an alert appears with the cost in rupees, in English, then switch to Hindi.
3. Press **Switch room off**. The room goes back to normal and the saving appears on Savings.
4. **Heat in Walls**: explain R = (V_board − V_room) / I with the diagram on the page; show a circuit whose resistance is climbing.
5. **HVAC & Comfort**: inject "AC set too cold" in CR-201; the room drops to 17 °C, the alert appears, and "Reset AC to 24 °C" fixes it.
6. **Water & Pump**: inject a stuck float switch; the tank overflow alert stops the pump.
7. **Real Buildings**: open a real classroom, scroll through 2017 and move the sandbox sliders.
8. **Savings & Carbon**: finish on the full-year before vs after.

14. Questions we expect, and our answers

Q. How good are the models on real buildings?
On 2017, a year they never saw, the electricity baseline has a median hourly error of 20.8% (CV(RMSE)) and 57.1% of buildings pass ASHRAE Guideline 14, the standard energy auditors use to accept a baseline. Labs pass most often (73.6%); classrooms are hardest because their use follows a timetable the data set does not include. Real 2017 meters also contain real faults, which count as model error here, so the true accuracy is a little better.

Q. What did ECOGUARD find in the real buildings?
In the median building, 13.8% of the electricity is avoidable, and one building in four is above 24.9%. Classrooms waste the most (18.8%). The median building with a cooling meter spends 43.0% of its cooling while closed, and 47 buildings heat and cool at the same time for more than 100 hours a year.

Q. Is this real data?
Both. The models were trained and tested on real meters: 659 real classrooms, labs and offices from the open Building Data Genome 2 data set, two full years of hourly electricity, cooling, heating and water readings (5.5 million training rows). Our own block in Jammu is simulated, because no public data set has room-level occupancy, wiring voltage, earth leakage and water flow together, with every fault labelled. The simulator's numbers come from public sources (IMD, CEA, JPDCL, BEE). The detector does not know which one it is talking to: it receives the same JSON a real ESP32 node would send.

Q. Why simulate at all? Why not use a public dataset?
Public datasets (BDG2, CU-BEMS and others) have whole-building or floor meters, but none of them has occupancy, wiring voltage drop, earth leakage and water flow for the same rooms at the same time, and none of them marks which hours were waste. Without that ground truth you cannot say how many faults you caught. The simulator gives us a full year with every fault labelled, so we can measure recall and precision honestly. Our mentor suggested exactly this.

Q. What is the AI part?
Each room has a learned model of what it should be using for the number of people inside, the outside temperature and the time of day. Anything above that expected use, with nobody to justify it, is waste. On top of that come physics checks the model cannot learn on its own (wiring resistance, earth leakage, minimum night flow, pump state). The advice text is written from templates filled with numbers computed in code, so it can never invent a figure.

Q. Why Huber regression and not a neural network?
Two reasons. It is explainable: we can say exactly how many kW one extra person or one degree of heat adds. And it is robust: the training year already contains waste, and Huber loss stops those spikes from pulling the 'normal' line upwards. On held-out data its error was lower than ordinary least squares in all 8 rooms (Computer Lab: 0.317 kW vs 0.525 kW). We do use deep learning where it pays off: the LSTM day-ahead forecaster, trained across hundreds of real buildings, where there is enough data and nobody needs to read the model's coefficients.

Q. How do you know a room is empty?
From the room node's people count. In the real build that comes from a PIR sensor or the existing CCTV with a person-detection model, and the timetable is a fallback. The simulator gives both a planned headcount (timetable) and the actual one, because classes get cancelled and people stay late.

Q. What if the system switches off a room with people in it?
It cannot. The switch-off command is only sent when the people count is zero, after a 15-minute grace period, and only for non-essential loads. Anyone pressing 'Not waste' stops the action and teaches the model. In the full-year test there were 0 cases of a room being switched off with people inside.

Q. How does 'heat in the walls' work?
Every wire and joint has resistance. Current through resistance makes heat ($I^2 \times R$) that no meter bills separately. We measure the voltage at the distribution board and at the room, and the current. The difference divided by the current is the resistance of the wiring in between. When a joint loosens or corrodes, that resistance climbs slowly over days. We learn each circuit's normal value and warn when it is 60% higher. The energy is small; the real value is catching a fire risk before it becomes one.

Q. And earth leakage?
A leakage sensor (the same principle as an RCCB) measures current escaping to earth. A healthy circuit leaks almost nothing. When water from a hidden pipe leak soaks a wall, leakage on that floor rises. If at the same time water keeps flowing on that floor at night, we say the leak is probably inside the wall. That combination is our second novel check.

Q. Why is the pump the centre of the water side?
Because it is where energy and water meet. If the float switch sticks, the pump keeps lifting water that pours off the roof: water wasted and electricity wasted to lift it. If the sump is empty, the pump runs dry, burns electricity and damages itself. Our level sensors let ECOGUARD stop the pump in both cases.

Q. How did you get the before-vs-after numbers?
We ran the same year twice: same people, same weather, same power cuts and the same 520 faults. The first run had nobody watching. In the second, every reading went through the real backend and every command came back to the building. Electricity fell 19.2%, water 22.6%.

Q. Why not 100% savings?
Some waste happens before any detector can be sure (a room has to be empty for 30 minutes before we call it waste), some needs a plumber or electrician who takes time, and some is small enough that shutting a whole floor's valve would be worse than the drip. We chose to be right and safe over being fast.

Q. Which waste is hardest to catch?
Lights and fans left on in a classroom for a short gap between classes: the room is empty for less than 30 minutes, so we let it go on purpose (91% caught overall). Before we added the wall temperature sensor, an AC set too cold was the hardest (32% caught from power alone); with the sensor it is 94%. We keep thresholds high on purpose, because a false alert in an occupied room is the fastest way to make staff ignore the system.

Q. What does it cost to deploy?
Per room: one ESP32 board, a current sensor, a voltage sensor at each end of the circuit, a leakage sensor and a relay module, roughly ₹2,500 to ₹3,500. For water: a flow meter per floor and level sensors for tank and sump. The software runs on one small server or a Raspberry Pi on campus.

Q. Can it work without internet?
Yes. The backend runs on the campus network. Readings are small JSON messages every 15 minutes, and the room nodes keep working on their own if the server is down.

Q. What happens during a power cut?
Grid meters read zero, so energy rules pause. Water rules continue. After power returns we watch for loads coming back on in empty rooms (switches left on), and we do not blame inverter rooms for the battery recharge, which is the cost of the outage and not someone's mistake.

Q. How is this different from a smart meter app?
A smart meter tells you how much you used. ECOGUARD tells you which room wasted it, why, what it cost, and fixes it, and it sees waste a meter never shows: heat in wiring, leaks inside walls, and the pump wasting both resources at once.

Q. How does the system get better over time?
Every alert has 'Correct' and 'Not waste' buttons. Each tap is stored as a label and can be downloaded as a CSV. Retraining on those labels tunes the model to this building.